



Collections Data Types

Lecturer: SO Vengleab
Date: July 2026

What is Collections?

Collections store groups of **multiple values or objects**, where as **primitive types** store **a single, basic value**

List

[]

Ordered, mutable sequence of items. Allows duplicates.

Tuple

()

Ordered, immutable sequence. Faster & hashable.

Dictionary

{ : }

Key-value pairs, unordered (3.7+ ordered), mutable.

Set

{ }

Unordered, unique elements. Fast membership testing.



List

List

- **Ordered** items
- **Different** data types within same list
- **Mutable** (can be modified).
- **Indexed** access

```
[4]: numbers = [1, 2, 3, 4, 5]
     fruits = ['apple', 'banana', 'orange']
     mixed_list = [1, 'apple', True]
```

Access by index

```
[4]: numbers = [1, 2, 3, 4, 5]
     fruits = ['apple', 'banana', 'orange']
     mixed_list = [1, 'apple', True]
```

```
[5]: numbers[0]
```

```
[5]: 1
```

```
[6]: fruits[1]
```

```
[6]: 'banana'
```

Access index out of range

```
mixed_list[3]
```

```
-----
IndexError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 mixed_list[3]

IndexError: list index out of range
```

Common List Methods

.append(x)

Add x to end

.insert(i,x)

Insert x at index i

.remove(x)

Remove first x

.pop(i)

Remove & return item at i

```
[4]: fruits = ['apple', 'banana', 'orange']  
  
fruits.append('mango')  
fruits      # ['apple', 'banana', 'orange', 'mango']
```

```
[4]: ['apple', 'banana', 'orange', 'mango']
```

```
[5]: fruits.insert(1, 'kiwi')  
fruits      # ['apple', 'kiwi', 'banana', 'orange', 'mango']
```

```
[5]: ['apple', 'kiwi', 'banana', 'orange', 'mango']
```

```
[6]: fruits.remove('banana')  
fruits      # ['apple', 'kiwi', 'orange', 'mango']
```

```
[6]: ['apple', 'kiwi', 'orange', 'mango']
```

```
[7]: fruits.pop(0)      # 'apple'
```

```
[7]: 'apple'
```

```
[8]: fruits
```

```
[8]: ['kiwi', 'orange', 'mango']
```

Extra List Methods

.sort()

Sort **in-place**

.reverse()

Reverse **in-place**

.index(x)

Index of **first** x

.count(x)

Count **occurrences** of x

```
[8]: fruits
```

```
[8]: ['kiwi', 'orange', 'mango']
```

```
[9]: fruits.sort()  
fruits # ['kiwi', 'mango', 'orange']
```

```
[9]: ['kiwi', 'mango', 'orange']
```

```
[10]: fruits.reverse()  
fruits # ['orange', 'mango', 'kiwi']
```

```
[10]: ['orange', 'mango', 'kiwi']
```

```
[11]: fruits.index('mango') # 1
```

```
[11]: 1
```

```
[12]: fruits.count('kiwi') # 1
```

```
[12]: 1
```

```
[13]: len(fruits) # 3
```

```
[13]: 3
```

Dictionary

Dictionaries

- **Key-value** pairs used to store data.
- **Mutable** and unordered.

```
# Example of dictionaries
```

```
person = {'name': 'John', 'age': 25, 'gender': 'Male'}  
person
```

```
{'name': 'John', 'age': 25, 'gender': 'Male'}
```

Access by **key**

```
# Example of dictionaries
```

```
person = {'name': 'John', 'age': 25, 'gender': 'Male'}  
person|
```

```
{'name': 'John', 'age': 25, 'gender': 'Male'}
```

```
person["name"]
```

```
'John'
```

```
person["age"]
```

```
25
```

Access by **wrong key**

```
person["dob"]
```

```
-----  
KeyError
```

```
Traceback (most recent call last)
```

```
Cell In[12], line 1
```

```
----> 1 person["dob"]
```

```
KeyError: 'dob'
```

Common Dict methods for read data

.keys()

Returns a view object of all keys

.values()

Returns a view object of all values

.get(key)

Returns value of key (safe access)

.items()

Returns view of key-value pairs

```
person = {'name': 'John', 'age': 25, 'gender': 'Male'}  
person.keys()    # dict_keys(['name', 'age', 'gender'])
```

```
dict_keys(['name', 'age', 'gender'])
```

```
person.values()  # dict_values(['John', 25, 'Male'])
```

```
dict_values(['John', 25, 'Male'])
```

```
person.items()  
# dict_items([('name', 'John'), ('age', 25), ('gender', 'Male')])
```

```
dict_items([('name', 'John'), ('age', 25), ('gender', 'Male')])
```

```
person.get('age')    # 25
```

```
25
```

Common Dict method for change data

.update(obj)

Merge keys from another dict

.pop(key)

Removes and returns item at key

.clear()

Removes all elements from dict

```
[27]: person.update({'age': 26})
      person    # {'name': 'John', 'age': 26, 'gender': 'Male'}

[27]: {'name': 'John', 'age': 26, 'gender': 'Male'}

[28]: person.pop('gender')    # 'Male'
      person

[28]: {'name': 'John', 'age': 26}

[29]: len(person)    # 2

[29]: 2

[30]: person.clear()
      person    # {}
```



Tuple

Tuple

- **Similar** to lists but immutable (cannot be modified).
- Typically used to store **related** pieces of information together.

```
# Example of tuples  
coordinates = (10, 20)  
person_info = ('John', 25, 'Male')
```

```
coordinates[0]
```

```
10
```

```
person_info[2]
```

```
'Male'
```

Common Tuple Methods

.count(x)

Returns the number of times a specified value occurs in a tuple

.index(x)

Searches the tuple for a specified value and returns the position of where it was found

len(tuple)

Returns the number of items in the tuple

```
[11]: numbers = [0,1,2,3,3,4,5]  
      numbers.count(3)
```

```
[11]: 2
```

```
[12]: numbers.index(3)
```

```
[12]: 3
```

```
[13]: len(numbers)
```

```
[13]: 7
```



Set

Sets

- **Unordered** collection
- **Unique** items.
- **Mutable** items, but **elements must be immutable**(hashable).

Common Set Methods

.add(x)

Add element x to the set

.remove(x)

Remove x; raises error if not present

.discard(x)

Remove x if present; no error if missing

.pop()

Remove and return a random element

```
[28]: fruits = {'apple', 'banana', 'orange'}  
      citrus = {'orange', 'lemon'}
```

```
[29]: fruits.add('mango')  
      fruits
```

```
[29]: {'apple', 'banana', 'mango', 'orange'}
```

```
[30]: fruits.remove('banana')  
      fruits
```

```
[30]: {'apple', 'mango', 'orange'}
```

```
[31]: fruits.pop() # removes & returns a random element
```

```
[31]: 'apple'
```

```
[32]: fruits
```

```
[32]: {'mango', 'orange'}
```

```
[33]: fruits.discard('kiwi') # no error  
      fruits
```

```
[33]: {'mango', 'orange'}
```

```
[35]: fruits.discard('mango')  
      fruits
```

Common Set Methods

.difference(s)

Return elements in this set but not in s

.intersection(s)

Return intersection as a new set

.union(s)

Return union of sets as a new set

```
[40]: fruits = {'apple', 'banana', 'orange'}  
      citrus = {'orange', 'lemon'}
```

```
[41]: fruits.union(citrus)
```

```
[41]: {'apple', 'banana', 'lemon', 'orange'}
```

```
[42]: fruits.intersection(citrus)
```

```
[42]: {'orange'}
```

```
[43]: fruits.difference(citrus)
```

```
[43]: {'apple', 'banana'}
```

Common Set Methods

.clear()

Remove all elements from the set

len(set)

Return the number of elements

```
[54]: fruits = {'apple', 'banana', 'orange'}
```

```
[55]: fruits.clear()
```

Once set is cleared, the items will be gone

```
[57]: len(fruits)
```

```
[57]: 0
```

```
[58]: fruits = {'apple', 'banana', 'orange'}  
len(fruits)
```

```
[58]: 3
```

Practice - Foundation

List Practice

- **Create:** a list of 3 elements: `bread`, `milk`, `eggs`
- **Append** `'butter'` to the end of the list.
- **Insert** `'cheese'` at index 1.
- **Remove** `'bread'` from the list.
- **Sort** the list alphabetically.
- **Print**
 - The list
 - and length of the list using `len()`.

```
groceries = ['bread', 'milk', 'eggs']
```

Expected result:

```
[1]: ['butter', 'cheese', 'eggs', 'milk']
```

```
[2]: (length 4)
```

Dictionary Practice

- **Create** dictionary of {'title': 'Dune', 'author': 'Frank Herbert', 'year': 1965}
- **Access** the author using `book['author']`.
- **Add** a new key 'genre' with the value 'Sci-Fi'.
- **Update** the year to 2021 .
- **Print**: the dict.
- **Print** `.keys()` and `.values()`.
- **Remove** the 'year' key.
- **Print** `.keys()` and `.values()`.

```
: book = {'title': 'Dune', 'author': 'Frank Herbert', 'year': 1965}
```

Expected result:

```
[1] {'title': 'Dune', 'author': 'Frank Herbert', 'year': 2021, 'genre': 'Sci-Fi'}  
[2] dict_keys(['title', 'author', 'year', 'genre'])  
[3] dict_values(['Dune', 'Frank Herbert', 2021, 'Sci-Fi'])  
[4] {'title': 'Dune', 'author': 'Frank Herbert', 'year': 2021}
```

Tuple Practice

- **Access** the second movie using `movies[1]`.
- **Count** how many times 'Inception' appears using `.count()`.
- **Find** the index of 'Avatar' using `.index()`.
- **Print** the length using `len()`.

:

```
movies = ('Inception', 'Titanic', 'Inception', 'Avatar')
```

Expected result:

```
[1] 2 # count of inception
```

```
[2] 3 # Index of avatar
```

```
[3] 4 # Length of movies
```


Set Practice

- **Add** 'orange' to `your_fruits`.
- **Find** the union of `your_fruits` and `friend_fruits`, and print it.
- **Find** the intersection (fruits you both like).
- **Find** the difference (fruits only you like).
- **Print** the length of `your_fruits` using `len()`.

:

```
your_fruits = {'apple', 'banana', 'mango'}  
friend_fruits = {'banana', 'kiwi', 'grape'}
```

Expected result:

```
[1] {'orange', 'banana', 'grape', 'kiwi', 'mango', 'apple'} # union
```

```
[2] {'banana'} # intersect
```

```
[3] {'apple', 'mango', 'orange'} # different
```

```
[4] 4 # Length of your_fruits
```

Practice - Basic Collection Methods

Grocery List Basics & Reversing & Sorting

1

Grocery List Basics

- Append 'butter'; insert 'cheese' at index 1
- Print the list and its `len()`

```
groceries = ['bread', 'milk', 'eggs']  
  
# TODO: append, insert at 1, print + len()
```

Expected output:

1. ['bread', 'cheese', 'milk', 'eggs', 'butter']
2. 5

2

Reversing & Sorting

- `.sort()` ascending, then `.reverse()`
- Print the final list

```
scores = [88, 72, 95, 60, 83]  
  
# TODO: sort ascending, reverse, print
```

Expected output:

1. [60, 72, 83, 88, 95]
2. [95, 88, 83, 72, 60]

Movie Tracker & Coordinate Unpacking

3 Movie Tracker

- `.count('Inception')` and `.index('Avatar')`
- Print both results

```
movies = ('Inception', 'Avatar',  
         'Inception', 'Titanic')  
  
# TODO: count + index, print both
```

Expected output:

1. count = 2
2. index = 1

4 Coordinate Unpacking

- Calculate `sum_xz = index 0 + index 2`
- Print `sum_xz`

```
point = (10, 25, 50)  
  
# TODO: sum_xz = point[0] + point[2]; print
```

Expected output:

`sum_xz = 60`

Student Profile & Updating Prices

5

Student Profile

- Print `['name']`; add `'gpa' = 3.8`
- Use `.get('scholarship', 'No')`

```
student = {'name': 'Alice', 'age': 20,  
          'major': 'CS'}  
  
# TODO: print name, add gpa, .get scholarship
```

Expected output:

1. Alice
2. scholarship -> No

6

Updating Prices

- `.update()` apple→1.5; `.pop('banana')`
- Print `.keys()`

```
prices = {'apple': 1.2, 'banana': 0.5,  
         'orange': 0.8}  
  
# TODO: update, pop, print keys()
```

Expected output:

1. dict_keys(['apple', 'orange'])
2. apple is now 1.5

Favorite Colors & Club Membership

7 Favorite Colors

- `.add('red'); .discard('blue')`
- Print the set and its count

```
colors = {'red', 'blue', 'green'}

# TODO: add, discard, print set + count
```

Expected output:

1. `{'green', 'red'}`
2. `Count: 2`

8 Club Membership

- **Intersection:** students in both clubs
- **Union:** all unique students

```
art_club = {'Alice', 'Bob', 'Charlie'}
drama_club = {'Bob', 'David', 'Charlie'}

# TODO: intersection, union
```

Expected output:

1. `intersect = {'Bob', 'Charlie'}`
2. `union = {'Alice', 'Bob', 'Charlie', 'David'}`